

Bokningssystem

Projektdokumentation

*En modern och responsiv plattform för gympassbokning,
byggd med React och ASP.NET Core Web API.*

1. Projektöversikt

Bakgrund

Core Gym Club AB är en gymkedja i Haninge som erbjuder gruppträning, personlig träning och öppen gymtid. Företaget efterfrågade ett digitalt bokningssystem som gör det möjligt för medlemmar att boka gympass, se scheman och hantera sina medlemskap. Målet är att bygga en modern, responsiv och användarvänlig bokningsplattform som betjänar både medlemmar och administratörer på ett effektivt sätt.

2. Projektmål

- Utveckla ett modernt gymbokningssystem
- Låta användare registrera sig och logga in
- Låta medlemmar se och boka träningssessioner
- Låta medlemmar avboka bokningar
- Låta administratörer hantera gymklasser och scheman
- Leverera en responsiv applikation för dator, surfplatta och mobil

3. MVP-omfattning

Användarfunktioner

- Registrera konto
- Logga in och logga ut
- Se kommande träningssessioner
- Boka en träningssession
- Avboka en bokning
- Visa profilinformation

Administratörsfunktioner

- Skapa träningssessioner
- Redigera träningssessioner
- Ta bort träningssessioner
- Hantera scheman

4. Agilt arbetssätt

Utvecklingsteamet använde agil metodik under projektet. Jira användes för sprintplanering och uppgiftshantering med veckovisa MVP-leveranser. Arbetet delades upp i mindre uppgifter för att göra utvecklingen tydlig, testbar och lättare att följa upp.

Exempel

- Backlog
- Att göra
- Pågår
- Testning

- Klar

5. Git-arbetsflöde

Git och GitHub användes för versionshantering med en funktionsgren-strategi. Färdiga funktioner slogs ihop med main-grenen via pull requests, vilket gav möjlighet till kodgranskning och minskade risken för konflikter före integration.

Exempel

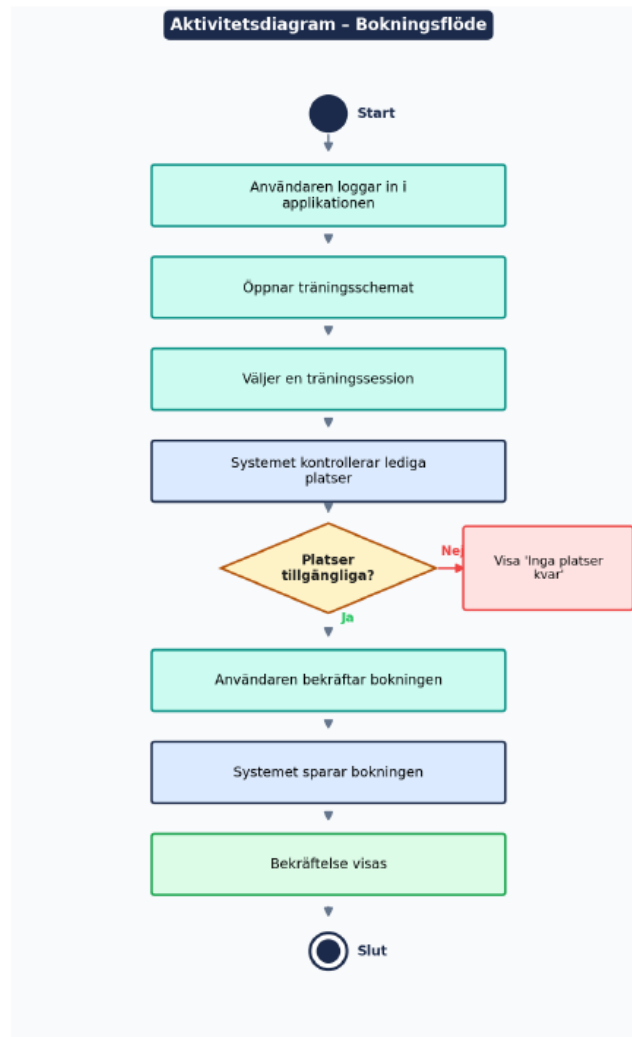
- feature/login
- feature/booking-system
- feature/admin-dashboard
- feature/profile-page

6. Användarberättelser

ID	Roll	Jag vill...	Så att...
US-01	Användare	skapa ett konto	jag kan använda bokningssystemet
US-02	Användare	logga in	jag kan hantera mina bokningar
US-03	Användare	logga ut	mitt konto förblir säkert
US-04	Medlem	se kommande gymklasser	jag kan välja ett lämpligt pass
US-05	Medlem	boka ett pass	jag kan reservera min plats
US-06	Medlem	avboka en bokning	jag kan frigöra min reserverade plats
US-07	Medlem	se mina kommande bokningar	jag kan hantera mitt schema
US-08	Administratör	skapa träningssessioner	medlemmar kan boka dem
US-09	Administratör	redigera träningssessioner	felaktig information kan uppdateras
US-10	Administratör	ta bort sessioner	inställda klasser tas bort från schemat

7. Aktivitetsdiagram - Bokningsflöde

Aktivitetsdiagrammet nedan modellerar det steg-för-steg-flöde en medlem följer för att boka ett gympass, inklusive systemets beslutspunkt som kontrollerar tillgängliga platser.



Figur 1 – Aktivitetsdiagram: Bokningsflöde för Medlem

Genomgång

Start: Flödet börjar när en registrerad medlem initierar en sessionsbokning.

Inloggning: Användaren autentiserar sig via ASP.NET Core/JWT-baserad autentisering.

Öppna schema: Medlemmen navigerar till träningsschema-sidan.

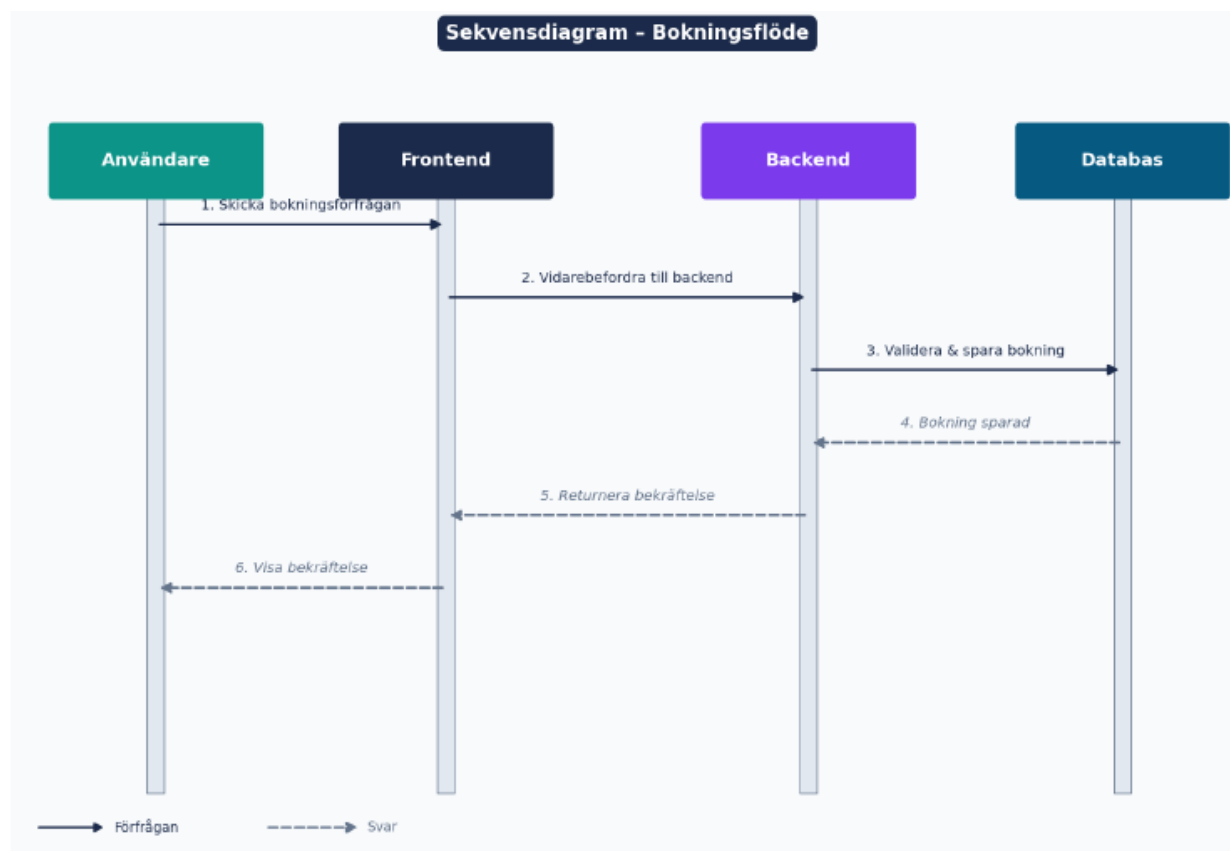
Välj session: Medlemmen väljer en önskad träningssession.

Tillgänglighetskontroll: Systemet kontrollerar återstående platser i databasen.

Beslut - Platser tillgängliga?: Ja: användaren bekräftar, systemet sparar bokningen och en bekräftelse visas. Nej: systemet visar meddelandet Inga platser kvar.

8. Sekvensdiagram - Bokningsflöde

Sekvensdiagrammet illustrerar det kronologiska meddelandeutbytet mellan användaren, frontend (React), backend (ASP.NET Core Web API) och databasen (SQLite) under en bokningsoperation.



Figur 2 - Sekvensdiagram: Bokningsflöde för medlem

Genomgång

Skicka bokningsförfrågan: Användaren trycker på Boka i gränssnittet.

Vidarebefordra till backend: React skickar ett API-anrop till ASP.NET Core Web API.

Validera och spara: Backend kontrollerar platsantal och skriver bokningen till databasen.

Bokning sparad: Databasen bekräftar skrivningen.

Returnera bekräftelse: Backend skickar ett framgångssvar till frontend.

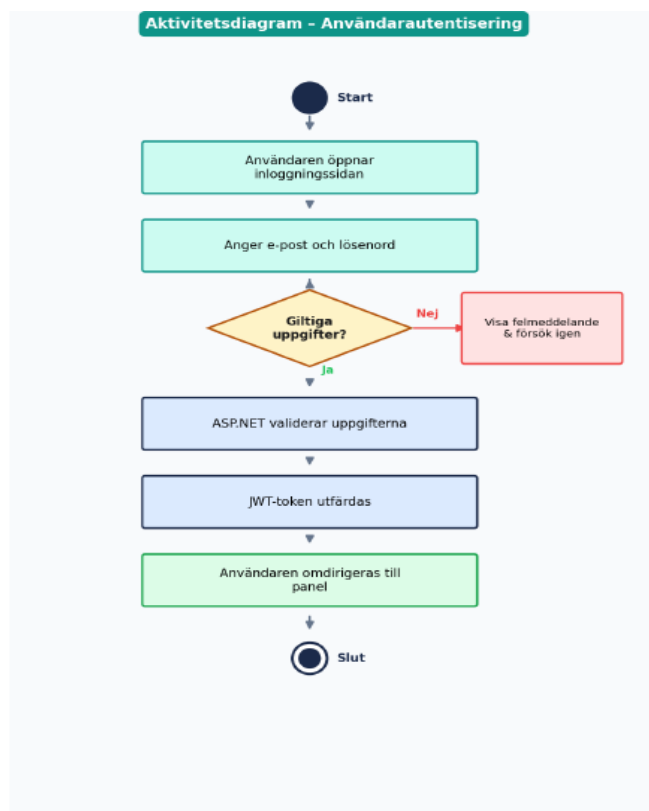
Visa bekräftelse: Frontend visar ett bekräftelsemeddelande för användaren.

9. Användarautentiseringsmodul

Autentisering hanteras av backend i ASP.NET Core. Modulen stöder e-post/lösenordsregistrering, inloggning, tokenbaserad sessionshantering och säker utloggning. En JWT-token utfärdas vid lyckad inloggning och används för efterföljande autentiserade API-förfrågningar.

9.1 Aktivitetsdiagram - Användarautentisering

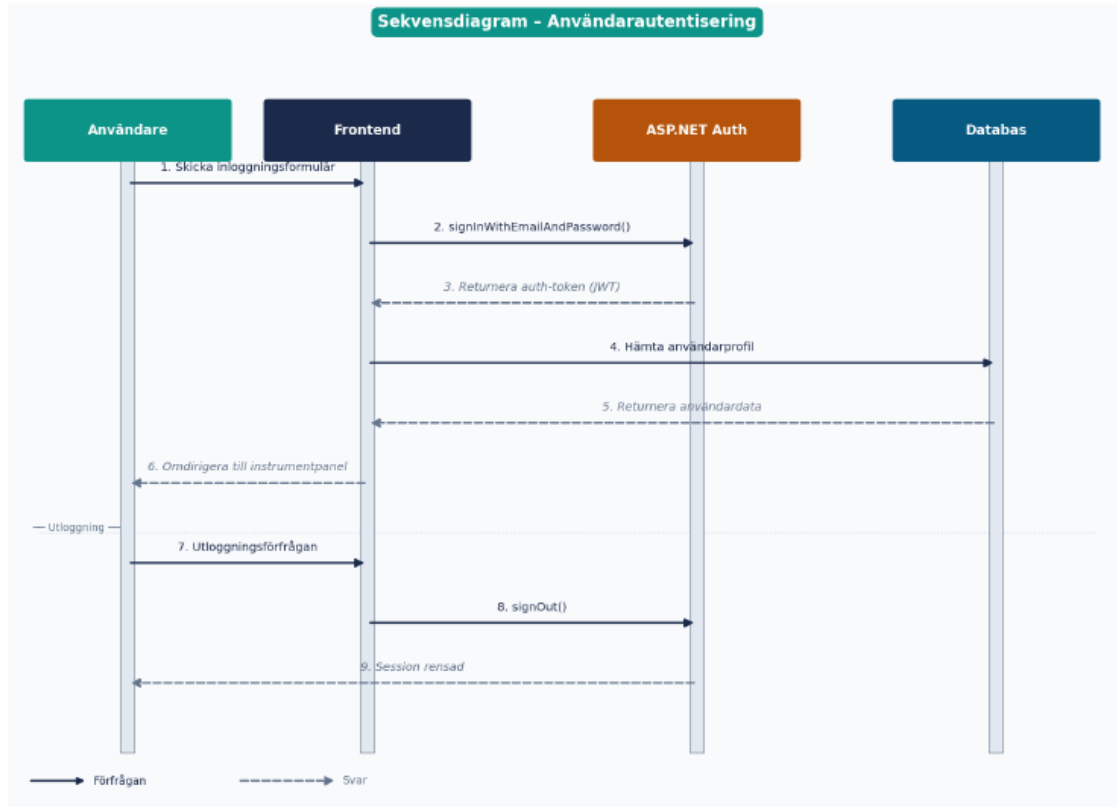
Diagrammet nedan modellerar inloggnings- och valideringsflödet, inklusive förgreningslogik för ogiltiga inloggningsuppgifter.



Figur 3 – Aktivitetsdiagram: Användarautentiseringsflöde

9.2 Sekvensdiagram - Användarautentisering

Detta diagram visar meddelandeutbytet från inloggningsformulärets inskickning via backend-validering och tokenutfärdande till omdirigering av panelen, samt utloggningssekvensen.



Figur 4 – Sekvensdiagram: Användarautentiseringsflöde

Autentiseringsgenomgång

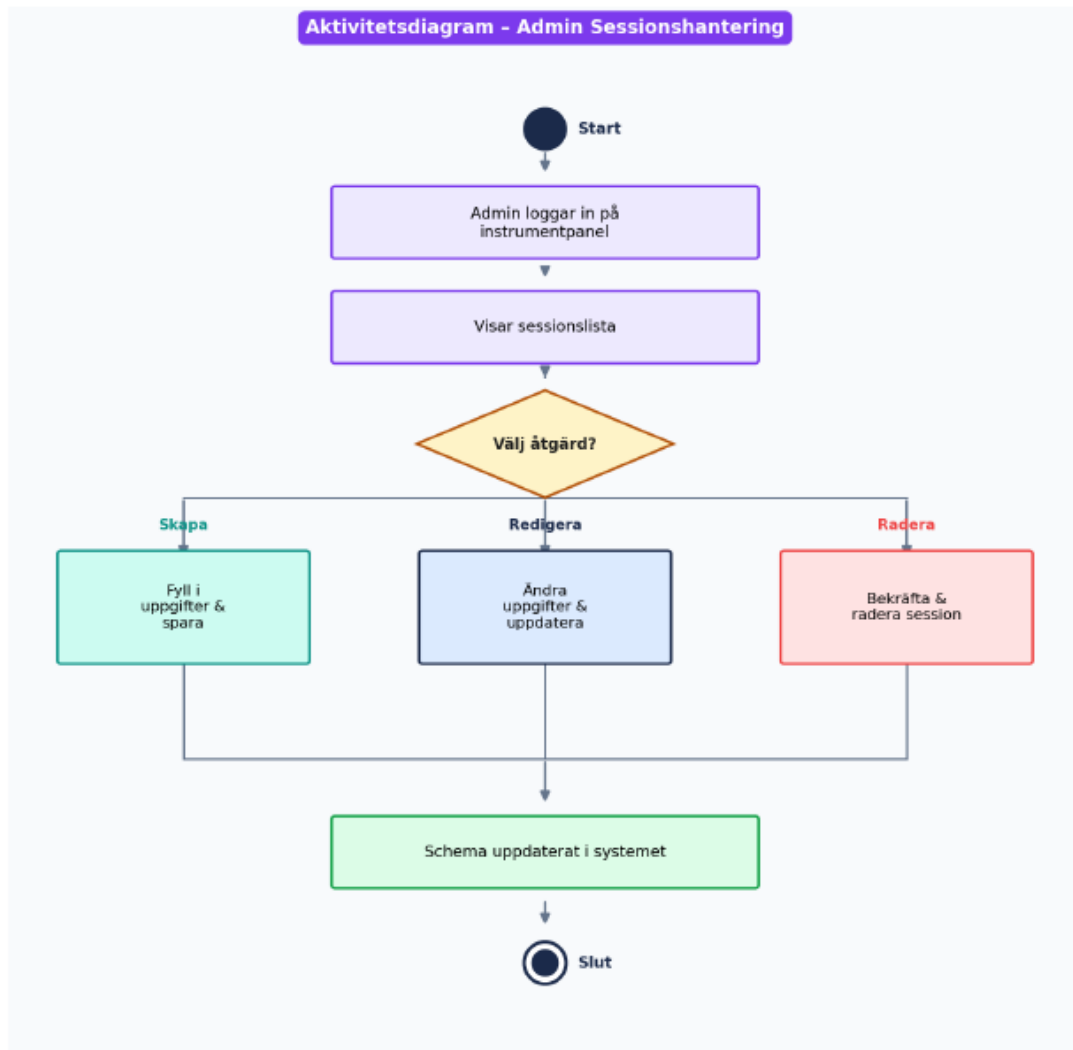
1. Användaren öppnar inloggningssidan och skickar in e-post/lösenord.
2. Frontend skickar uppgifterna till backend via ett API-anrop, till exempel POST /api/auth/login.
3. ASP.NET Core Web API validerar uppgifterna mot databasen och returnerar en JWT-token vid lyckad inloggning.
4. Frontend sparar token enligt vald säkerhetslösning och använder den vid skyddade API-anrop.
5. Användaren omdirigeras till medlemsinstrumentpanelen.
6. Vid utloggning rensas token/session på klientsidan och användaren skickas tillbaka till inloggningssidan.

10. Adminsessionshanteringsmodul

Adminmodulen ger autentiserade administratörer kontroll över träningssessioner: skapa nya sessioner, redigera befintliga och ta bort inställda sessioner. Alla ändringar sparas direkt i databasen.

10.1 Aktivitetsdiagram - Admin Sessionshantering

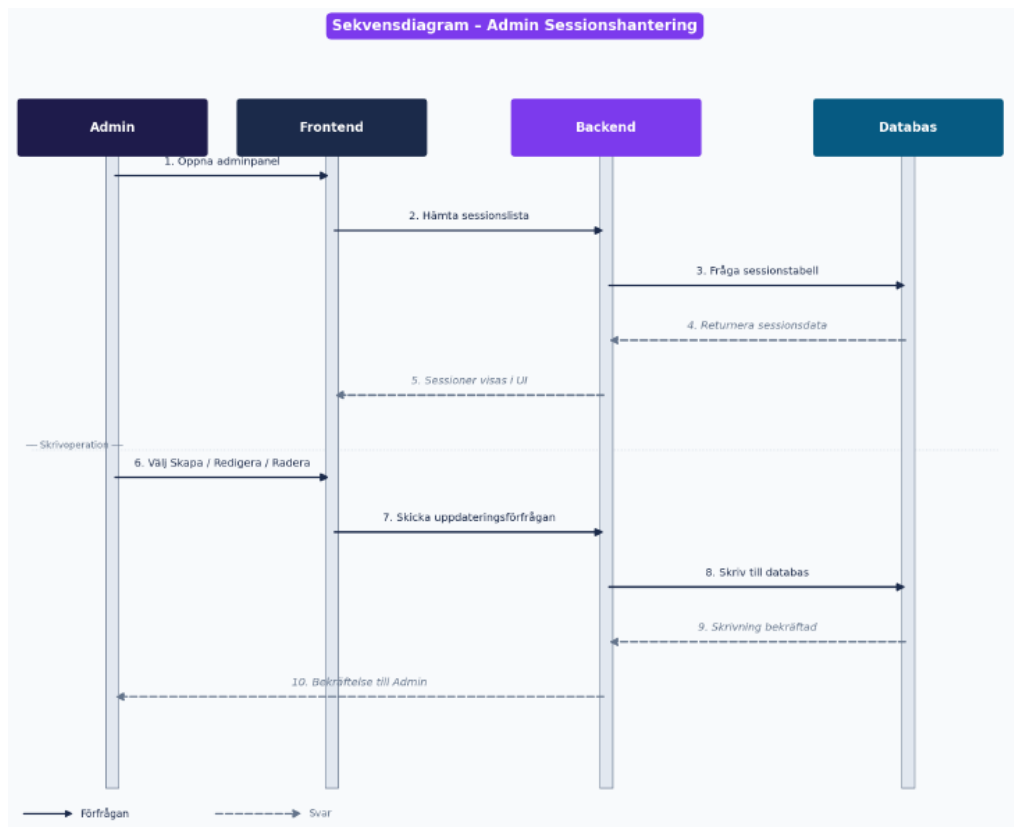
Det förgrenade aktivitetsdiagrammet nedan visar de tre vägarna en admin kan ta efter att ha sett sessionslistan: skapa, redigera eller radera.



Figur 5 - Aktivitetsdiagram: Admin Sessionshantering

10.2 Sekvensdiagram - Admin Sessionshantering

Detta diagram visar interaktionen mellan admin, frontend, backend och databasen - från inläsning av sessionslistan till bekräftelse av en skapa-, redigera- eller radera-operation.



Figur 6 - Sekvensdiagram: Admin Sessionshantering

Adminflödesgenomgång

1. Admin öppnar adminpanelen via en autentiserad rutt.
2. Frontend hämtar den aktuella sessionslistan via backend.
3. Sessioner renderas i adminens gränssnitt.
4. Admin väljer skapa, redigera eller radera.
5. Uppdateringsförfrågan skickas till backend.
6. Backend skriver ändringen till sessionstabellen i databasen.
7. Databasen bekräftar skrivningen och admin får en bekräftelse i gränssnittet.

11. Teknisk stack

Lager	Teknik
Frontend	React
Backend	ASP.NET Core Web API
Databas	SQLite
ORM	Entity Framework Core
Autentisering	ASP.NET Core-baserad autentisering med JWT
API-dokumentation/Testning	Swagger
Versionshantering	Git + GitHub
Grenhantering	Funktionsgrenar + Pull Requests
Projekthantering	Jira
Design	Responsiv webbdesign
Utvecklingsverktyg	Visual Studio + VS Code
Programmeringsspråk	C#, JavaScript, HTML, CSS
Publicering / Hosting	Koden publiceras på GitHub. Frontend kan publiceras via GitHub Pages, medan backend kräver separat hosting, till exempel Azure, Render eller annan servermiljö.

12. Responsiv design

Applikationen utformades för att fungera smidigt på olika enhetstyper: mobil, surfplatta och dator. Responsiva layouter implementerades för att förbättra användbarheten och bibehålla en konsekvent upplevelse på olika skärmstorlekar.

- Mobila enheter via responsiv React-webbdesign
- Surfplattor
- Webbläsare på dator

13. Utmaningar

Utmaning	Lösning
Konfigurationsproblem för miljövariabler	Förbättrade initialiseringsskript och tydligare miljöinställningar.
Problem vid API-testning i Swagger	Noggrann testning och kontroll av API-konfigurationer.
Hantering av tokenbaserad autentisering	JWT-flödet testades och skyddade routes kontrollerades.
Offline-tillstånd och API-fel	Felhantering och tydligare meddelanden lades till i gränssnittet.

14. Veckovis MVP-progression

Sprint	Tema	Viktiga leveranser
Vecka 1	Grund	Projektkonfiguration, ASP.NET Core Web API-projekt, databasstruktur och autentiseringssystem.
Vecka 2	Kärnbokning	Bokningssystem, sessionslistning, filtrering och användarprofilfunktioner.
Vecka 3	Admin och finslipning	Adminpanel, skapa/redigera/ta bort sessioner och förbättrad responsiv layout.
Vecka 4	Leverans	Testning, felrättning, kodpublicering på GitHub och projektdokumentation.

15. Reflektioner

Projektet fördjupade teamets förståelse för modern fullstackutveckling. Viktiga lärdomar inkluderade:

- Agil utveckling och sprinthantering med Jira
- Git-arbetsflöde med funktionsgrenar, pull requests och kodgranskning
- ASP.NET Core Web API, Entity Framework Core och SQLite
- JWT-baserad autentisering och skyddade API-anrop
- Responsiv design för mobil, surfplatta och dator
- Felsökning och testning av API:er med Swagger
- MVP-planering och iterativ funktionsprioritering

Teamet förstärkte också vikten av kontinuerlig testning, noggrann dokumentation, disciplinerad versionshantering och inkrementell MVP-leverans för snabba återkopplingscykler.

16. Slutsats

Projektet levererade ett fungerande MVP-bokningssystem för Core Gym Club AB. Applikationen byggdes med agila metoder, Git-arbetsflöden och veckovisa MVP-leveranser i enlighet med uppgiftskraven.

Medlemmar kan:

- Registrera sig och logga in säkert
- Se tillgängliga gympass
- Boka och avboka träningssessioner
- Visa och hantera sitt kommande schema

Administratörer kan:

- Skapa, redigera och ta bort träningssessioner
- Uppdatera och hantera träningsschemat